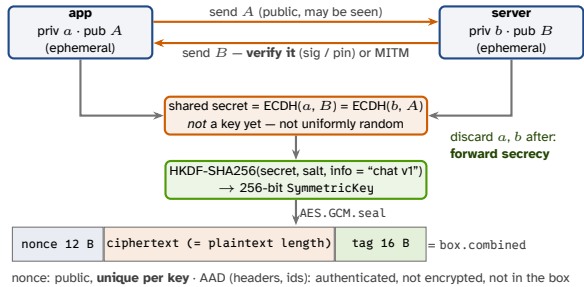


In one line: Symmetric AEAD (AES-GCM, ChaCha20-Poly1305) encrypts *and* authenticates bulk data under one key and a **nonce never repeated under that key**; asymmetric keys *agree* on that key (ECDH → HKDF, or a KEM inside HPKE) and *sign* (ECDSA, Ed25519, ML-DSA). CryptoKit ships vetted primitives with safe defaults — the job is choosing them, managing keys and nonces, and **never inventing** a primitive, mode or protocol.

Agree → derive → seal (the shape of TLS 1.3)



How it works

- AEAD** = confidentiality + integrity + authenticity; open throws on any flipped bit or wrong AAD. **AES-GCM**: 128/192/256-bit key. **ChaCha20-Poly1305**: 256-bit key, $\sim 3\times$ faster than AES on chips without AES hardware. Both: 12-byte nonce, 16-byte tag.
- Nonce reuse** under one key: $C_1 \oplus C_2 = P_1 \oplus P_2$, and GCM gives away its **hash key** → trivial forgeries under that key (Poly1305: the one-time key repeats). A repeated nonce on *decrypt* is harmless.
- ECDH** output is not uniformly random: HKDF **extract** (salt; none = zeros), then **expand** with info — one purpose per key, $\leq 255 \times$ hash length. Ephemeral pairs, discarded → forward secrecy.
- ECDSA** $s = (h + xr)/k$: the same k twice gives $k = (h_1 - h_2)/(s_1 - s_2)$, then the private key $x = (sk - h)/r$. RFC 6979 k and EdDSA are deterministic — **CryptoKit randomizes both**: new bytes every call for the same key + message.
- HPKE** (RFC 9180) = KEM → KDF → AEAD to a public key; nonce = base nonce XOR sequence number, so **open in the order sealed**: no loss or reordering, no replay protection, no forward secrecy if the recipient key leaks.

AEAD limits per key

construction	per (key, nonce)	random nonces
AES-256-GCM	~ 64 GB	2^{32} messages
ChaCha20-Poly1305 (IETF)	256 GB	2^{32} messages
XChaCha20 (not in CryptoKit)	$\sim 2^{64}$ B	no practical limit

1 MB messages, 2^{-32} attack chance: AES-GCM $\sim 2^{32}$ encryptions per key, ChaCha 2^{48} . Then rekey; chunk huge messages.

Security strength — equal rows

bits	AES	elliptic curve	RSA	hash
128	AES-128	P-256, X25519, Ed25519	3072	SHA-256
192	AES-192	P-384	7680	SHA-384
256	AES-256	P-521	15360	SHA-512

ML-DSA-44 / 65 / 87 = NIST levels 2 / 3 / 5 ($3 \approx \text{AES-192}$, $5 \approx \text{AES-256}$). No RSA in CryptoKit → SecKey.

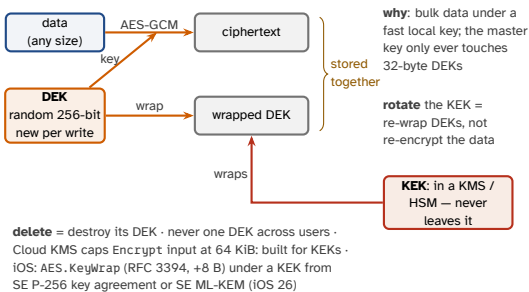
Example — the CryptoKit calls

```
let mine = Curve25519.KeyAgreement.PrivateKey() // ephemeral
let key = try mine.sharedSecretFromKeyAgreement(with: theirPub)
    .hkdfDerivedSymmetricKey(using: SHA256.self, salt: salt,
    sharedInfo: Data("chat v1".utf8), outputByteCount: 32)
let box = try AES.GCM.seal(msg, using: key, authenticating: hdr)
let wire = box.combined! // random 12 B nonce | ct | tag
let plain = try AES.GCM.open(.init(combined: wire), using: key,
    authenticating: hdr)

let signer = P256.Signing.PrivateKey() // SecureEnclave.P256: same
let sig = try signer.signature(for: msg) // SHA-256, randomized
let ok = signer.publicKey.isValidSignature(sig, for: msg)
let jws = sig.rawRepresentation // r/s 64 B; X.509: DER
// HPKE, hybrid post-quantum (iOS 26)
let suite = HPKE.Ciphersuite.XwingMLKEM768X25519_SHA256_AES_GCM_256
let sk = try XwingMLKEM768X25519.PrivateKey()
var tx = try HPKE.Sender(recipientKey: sk.publicKey,
    ciphersuite: suite, info: info)
let c1 = try tx.seal(m1) // send tx.encapsulatedKey once
var rx = try HPKE.Recipient(privateKey: sk, ciphersuite: suite,
    info: info, encapsulatedKey: tx.encapsulatedKey)
let p1 = try rx.open(c1) // open in the order sealed
```

Storing keys: P-256/384/521 → SecKey via x963Representation; Curve25519 and SymmetricKey bytes → a generic-password Keychain item.

Envelope encryption



CryptoKit by release

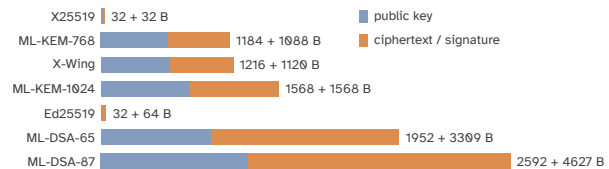
iOS	added
13	SymmetricKey(size:) · AES.GCM · ChaChaPoly · SHA256/384/512 · HMAC · Curve25519, P256/384/521 agreement + signing · SecureEnclave.P256 · Insecure.MD5 / .SHA1 (the namespace is the warning)
14	HKDF: extract, expand, deriveKey
15	AES.KeyWrap (RFC 3394)
17	HPKE (Sender, Recipient; 4 classical suites), KEM
26	MLKEM768/1024, MLDSA65/87, XwingMLKEM768X25519 (+ HPKE suite), SHA-3; the ML types also under SecureEnclave
27	one-time ML-KEM / X-Wing private keys; seal/open(inplace:) on spans

Secure Enclave

Key types: P256 (signing, agreement), since iOS 26 also MLKEM768/1024 and MLDSA65/87 — nothing symmetric, no one-time keys. Born inside, never leaves: dataRepresentation is an *encrypted block* only the same SE can restore (store it in the Keychain). Default access control AfterFirstUnlockThisDeviceOnly, no flags.

Post-quantum — why, and the cost

Ciphertext recorded today can fall to a future quantum computer, so **key agreement moves first**: FIPS 203 (ML-KEM), 204 (ML-DSA), August 2024. A **KEM** encapsulates to a public key → (secret, ciphertext); only the private key decapsulates, in CryptoKit straight to a SymmetricKey. **Hybrid** = classical + PQ, safe while *either* holds: **X-Wing** = ML-KEM-768 + X25519 (aims at 128-bit, 768 for margin); signatures: ML-DSA || ECDSA, verify both.



One-time ML-KEM / X-Wing keys (iOS 27): faster, consumed by decapsulate, never saved.

Traps

- ECB** (equal blocks → equal ciphertext); **CBC** without a MAC → padding oracles. Use AEAD.
- Fixed or reset nonce** (a counter restarting after reinstall; two devices, one key). seal(inplace:) has no random default.
- Hard-coded keys** — strings finds them. **Password as a key** → slow KDF first: CCKeyDerivationPBKDF (OWASP $\geq 600\,000 \times \text{HMAC-SHA256}$) or Argon2id.
- Tag compared after conversion** to Data / [UInt8] / hex String with == — early exit leaks timing. swift-crypto's MAC == and isValidAuthenticationCode are constant-time.
- Signature bytes as an ID** or test oracle — they differ every call; verify instead. ES256 JWS = the 64-byte raw form, not DER.
- One key, two jobs** → separate HKDF info. **Unauthenticated ECDH** = agreeing with the attacker.
- Key bytes from GameplayKit ("not cryptographically robust") or a seeded test RNG.

Remember

Agree (ECDH or KEM) → derive (HKDF) → seal (AEAD, unique nonce) → sign what must be proven; hybrid PQ for secrets that must outlive a quantum computer. Never roll your own.